Requirements

Functional Requirements

Each requirement below names the class that implements it, the GoF pattern(s) it draws on, and the test file that verifies it. All test files live under tests/

Core Engine

1. Manage reusable workflow definitions: store activity descriptions, dependencies, required roles, and process rules. Validate a definition before starting an execution.
    Implementation: WorkflowDefinition, WorkflowDefinition::validate(), WorkflowBuilder (Builder, Composite)
    Test File: test_builder.cpp
2. Create independent workflow instances from a definition. Each instance carries its own data, participants, progress, and history.
    Implementation: WorkflowDefinition::createInstance(), WorkflowInstance (Prototype)
    Test File: test_prototype.cpp
3. Represent individual activities and groups of activities as a uniform tree. Support sequential and parallel work.
    Implementation: WorkItem, TaskItem, StageItem (Composite, Template Method)
    Test File: test_composite.cpp
4. Enforce dependencies: a work item becomes available only when all items it depends on are settled.
    Implementation: WorkItemSpec::dependencies(), FlowController::releaseEligible() (Observer, Iterator)
    Test File: test_dependencies.cpp
5. Track each work item's lifecycle state and permit only valid operations for that state.
    Implementation: WorkItemState + 9 concrete states (State)
    Test File: test_decorator.cpp
6. Enforce role-based permissions: an actor may only start, submit, complete, approve, or reject an item if they hold the required role.
    Implementation: SecureWorkItem, WorkItemSpec::requiredRole(), reviewerRole() (Decorator)
    Test File: test_state.cpp
7. Allow routing policy to be configured per workflow and per source item, without editing work-item classes.
    Implementation: RoutingStrategy, DependencyRouting, OutcomeTableRouting, DataValueRouting (Strategy (routing))
    Test File: test_strategy_routing.cpp
8. Allow assignment policy to be configured per workflow, without editing work-item classes.

Implementation: AssignmentStrategy, RoleBasedAssignment, DataDrivenAssignment (Strategy (assignment))
Test File: test_strategy_assignment.cpp

9. Support reviewed items: worker submits, reviewer approves or rejects. Rejections return to the worker up to a configured limit.
   Implementation: UnderReviewState, WorkItemSpec::reviewRequired(), maxRetries() (State)
   Test File: test_state.cpp
10. Enforce deadlines. When a deadline lapses, either escalate or complete with a configurable default outcome.
    Implementation: DeadlineMonitor, WorkItem::expire(), FlowController (Observer, State)
    Test File: test_deadline.cpp
11. Support multi-level escalation with configurable chains. An escalated item is routed to the approver role resolved by the chain.
    Implementation: EscalationHandler, RoleEscalationHandler, EscalatedState (Chain of Responsibility, State)
    Test File: test_chain.cpp
12. Record every state change in an append-only audit log.
    Implementation: WorkflowEvent, AuditObserver, AuditLog (Observer)
    Test File: test_observer.cpp
13. Support undo of the most recent reversible action. Undo restores prior state and appends a compensating record.
    Implementation: InstanceCommand, InstanceMemento, ActionHistory (Command, Memento)
    Test File: test_command_undo.cpp
14. Support grouping several actions into one all-or-nothing command.
    Implementation: MacroCommand (Composite, Command)
    Test File: test_command_undo.cpp
15. Integrate with external systems through a uniform adapter interface, so vendor changes do not affect the engine.
    Implementation: ExternalConnector, ConnectorRegistry (Adapter)
    Test File: test_adapter.cpp
16. Provide a single façade for the UI and tests to interact with the engine.
    Implementation: Tividy (Façade)
    Test File: test_end_to_end_banking.cpp, test_end_to_end_telecom.cpp
17. Publish state changes as events so independent observers can react without coupling to work items.
    Implementation: EventPublisher, EventObserver, FlowController (Observer)
    Test File: test_observer.cpp

Extensibility

1. Allow new industries to be added as plug-in packs without editing engine classes.
   Implementation: IndustryPack, Tividy::installPack() (Abstract Factory)
   Test File: test_both_industries.cpp

2. Allow workflows to be defined by a Director using a standard builder, separated from the engine.
    Implementation: WorkflowDirector, WorkflowBuilder, StandardWorkflowBuilder (Builder)
    Test File: test_builder.cpp

Industries

1. Execute a banking transaction dispute workflow end-to-end, including an exception path.
    Implementation: BankingPack, DisputeDirector, banking::* tasks (Engine-level used by industries, integrating patterns such as Adapter, Strategy, Chain of Responsibility)
    Test File: test_end_to_end_banking.cpp
2. Execute a telecommunications service provisioning workflow end-to-end, including an exception path.
    Implementation: TelecomPack, ServiceActivationDirector, telecom::* tasks (Engine-level used by industries, integrating patterns such as Adapter, Strategy, Chain of Responsibility)
    Test File: test_end_to_end_telecom.cpp
3. Both industries run on the same unmodified engine and appear in the same running binary.
    Implementation: Tividy, both packs installed at startup (Abstract Factory)
    Test File: test_both_industries.cpp

User Interface

1. Provide an interactive terminal UI that lets a user start an instance, inspect available work, take actions, view history, and observe routing and state changes.
    Implementation: Tui, Tui::run() (UI, no patterns apply directly)
    Test File: test_tui_scripted.cpp
2. Both industries are accessible from the same running UI, with industry-appropriate vocabulary.
    Implementation: ClientProfile, Tui::profileFor() (UI, no patterns apply directly)
    Test File: test_tui_scripted.cpp
3. The UI reports illegal actions as a controlled message, never crashing or silently ignoring.
    Implementation: CommandResult::failure(), Tui message rendering (Command, UI)
    Test File: test_tui_scripted.cpp

Non-Functional Requirements

1. Written in C++11, compiled with -std=c++11.
2. Build reproducible from a clean clone.
3. All engine classes documented with Doxygen.
4. Automated tests runnable with a single command.

Deferred Requirements

The following requirements were considered and deferred, with brief reasons supplied:

1. Multi-version workflow definition storage
   Reason: Single active definition per name sufficient for both industries
2. Graphical workflow editor
   Reason: Out of scope for the submission
3. Full BPMN compatibility
   Reason: Out of scope for the submission
4. Distributed execution
   Reason: Each Tividy runs in a single process
5. Production identity / authentication
   Reason: Participants are seeded by packs
6. Analytics observer
   Reason: Progress is computed on demand
7. Command-gatekeeper chain (second CoR)
   Reason: Action validation handled by states and ActionHistory
8. Configurable failure-retry policy
   Reason: Retry logic inline in FlowController